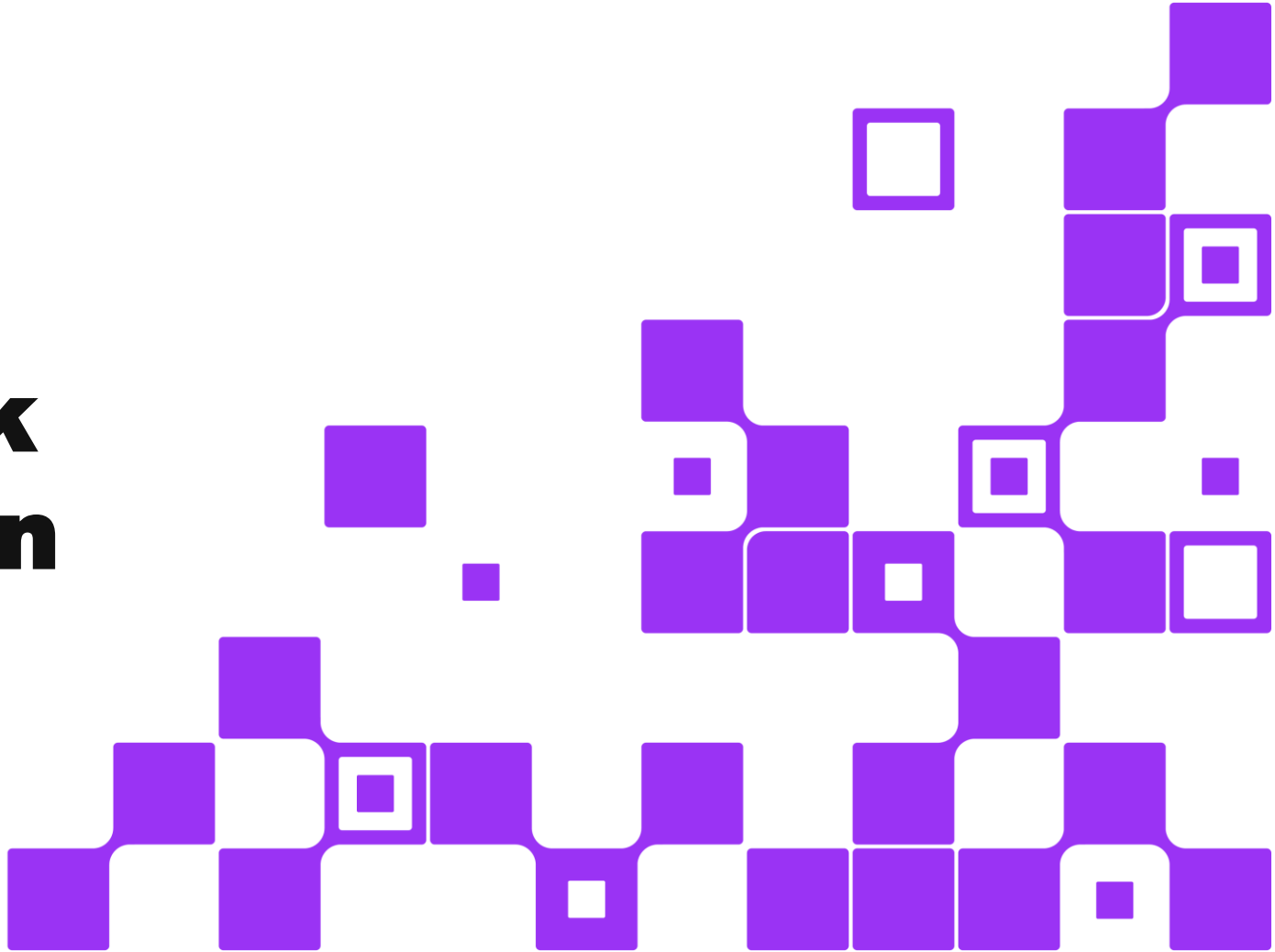
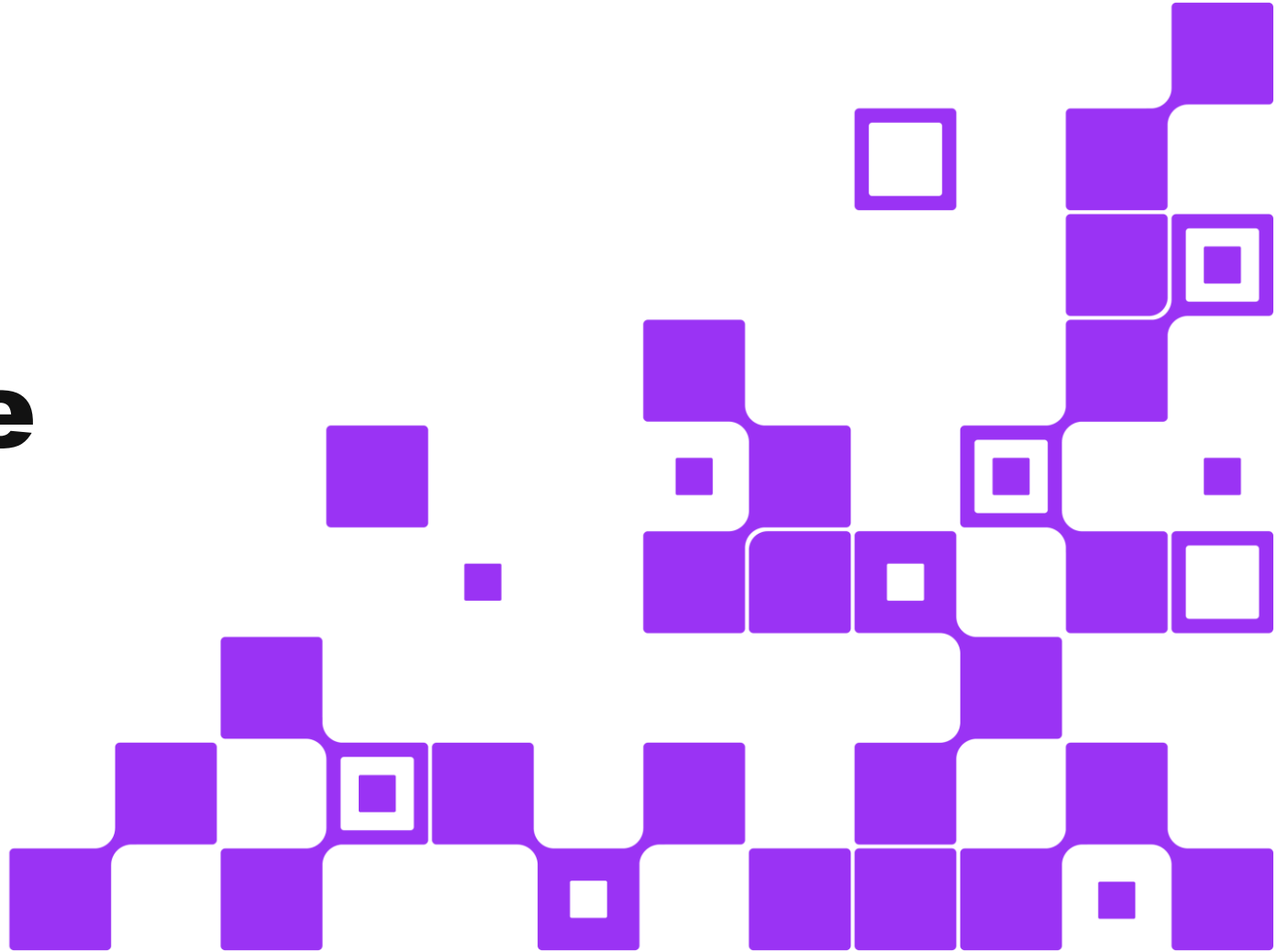
IThub

Maven
Lombok
Jackson



IThub

Apache Maven



Maven™

IThub

Apache Maven - это фреймворк для автоматизации сборки проектов на основе описания их структуры в файлах на языке POM (Project Object Model).

правильно подключать библиотеки

управлять версиями библиотек

выстроить структуру проекта

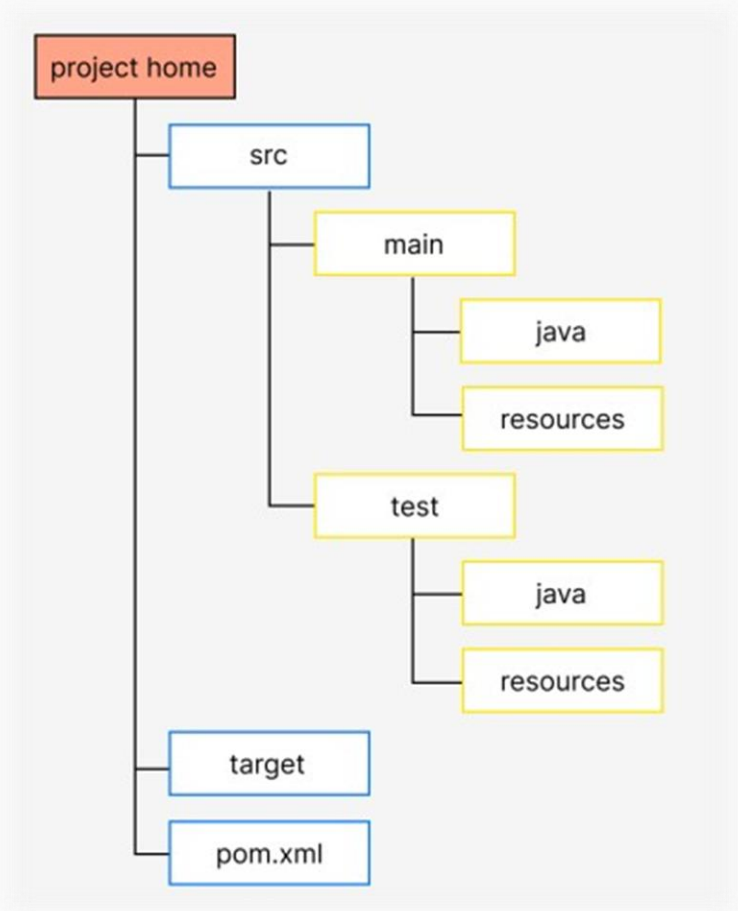
составить к нему документацию

Жизненный цикл

IThub

clean	удаляются все скомпилированные файлы из каталога target
validate	идет проверка, вся ли информация доступна для сборки проекта
compile	компилируются файлы с исходным кодом
test	запускаются тесты
package	упаковываются скомпилированные файлы (в jar, war и т.д. архив)
verify	выполняются проверки для подтверждения готовности упакованного файла
install	пакет помещается в локальный репозиторий. Теперь он может использоваться другими проектами как внешняя библиотека
site	создается документация проекта
deploy	собранный архив копируется в удаленный репозиторий

Типовая структура maven-проекта



IThub

Модульный maven проект

IThub

Пример структуры проекта с двумя модулями

- my-project
  - pom.xml
  - core
    - pom.xml
    - src/
      - main/
        - java/
        - resources/
      - test/
        - java/
        - resources/
  - app
    - pom.xml
    - src/
      - main/
        - java/
        - resources/
      - test/
        - java/
        - resources/



Корневой pom.xml (управляет модулями)

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">
   <modelVersion>4.0.0</modelVersion>

   <groupId>com.example</groupId>
   <artifactId>my-project</artifactId>
   <version>1.0-SNAPSHOT</version>
   <packaging>pom</packaging>

   <modules>
      <module>core</module>
      <module>app</module>
   </modules>

</project>
```

pom.xml модуля core

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
         <groupId>com.example</groupId>
         <artifactId>my-project</artifactId>
         <version>1.0-SNAPSHOT</version>
    </parent>

    <artifactId>core</artifactId>
    <packaging>jar</packaging>
 </project>
```

pom.xml модуля app

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
         <groupId>com.example</groupId>
         <artifactId>my-project</artifactId>
         <version>1.0-SNAPSHOT</version>
    </parent>

    <artifactId>app</artifactId>
    <packaging>jar</packaging>

     <dependencies>
          <dependency>
               <groupId>com.example</groupId>
               <artifactId>core</artifactId>
               <version>1.0-SNAPSHOT</version>
          </dependency>
    </dependencies>
  </project>
```

IThub

Что происходит при сборке

1. Вы запускаете команду в корневой директории:
   ```
   mvn clean install
   ```
2. Maven читает центральный pom.xml и видит, что проект состоит из модулей core и app.
3. Сначала собирается модуль core (если он не требует зависимостей от других модулей). Его артефакт (jar) устанавливается в локальный репозиторий Maven.
4. Затем собирается модуль app, который зависит от core. Maven находит нужную зависимость (уже установленную в репозиторий).
5. В результате получается готовый артефакт (например, jar или war) для каждого модуля.



Важно: Модулей может быть сколько угодно — 2, 3, 5 и более.
В корневом pom.xml просто перечисляйте их в секции <modules>.

IThub

pom.xml – это основной конфигурационный файл в проекте, который описывает его структуру, зависимости и настройки.

project - корневой элемент, содержащий всю информацию о проекте

groupId, artifactId, version — это обязательные элементы для идентификации содержимого проекта

properties — здесь указываются переменные проекта

dependencies - здесь определяются зависимости проекта - библиотеки, которые будут использоваться

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>
    <artifactId>JavaExamples</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <jar.plugin.version>2.4</jar.plugin.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter-engine</artifactId>
            <version>5.10.2</version>
            <scope>test</scope>
    </dependencies>
</project>
```

IThub

Семантическое версионирование

IThub




```
<groupId>org.example</groupId>
<artifactId>JavaExamples</artifactId>
<version>5.0.2</version>
```

Major Version — (основная версия) - когда выпускается версия, в которой появляются новые функции, из-за которых совместимость с предыдущими версиями может быть не гарантирована

Minor Version — (второстепенная версия) - когда выпускается версия, которая оптимизирует существующие функции или добавляет новые, полностью совместимую с предыдущими выпусками

Incremental Version \patch — (добавочная версия\патч) - когда выпускается версия, которая исправляет только ошибки и полностью совместима с предыдущей

Release Qualifiers

IThub

alpha, beta	Разработка этой версии находится на очень ранней стадии
snapshot	Для версии, над которой мы сейчас работаем. Это укороченная версия приложения, которая может измениться в любой момент
release	Эта версия программного обеспечения является окончательной и не подлежит изменению. Любые возможные улучшения этой версии будут выпущены с другим номером версии

Snapshot

IThub

```
<project ……>

  <groupId>sasha.example</groupId>
  <artifactId>my-lib</artifactId>
  <version>1.0-SNAPSHOT</version>

  </project>
```

Например, у нас есть maven-проект. Это просто какая-то вспомогательная библиотека с классами, которые содержат статические методы:

```
public class LogUtils {

    public static String getDebugMessage(String message) {
        return "[DEBUG] " + message;
    }

    public static String getInfoMessage(String message) {
        return "[INFO] " + message;
    }

    public static String getWarnMessage(String message) {
        return "[WARN] " + message;
    }
}
```

```
public class StringUtils {

    public static String toLowerCase(String str) {
        return str.toLowerCase();
    }

    public static String toUpperCase(String str) {
        return str.toUpperCase();
    }
}
```

mvn install

IThub

Run | my-lib [install]

my-lib [install]: At 03.10.2025 3 sec, 918 ms

```
[INFO] --- jar:3.4.1:jar (default-jar) @ my-lib ---
[INFO] Building jar: E:\my-lib\target\my-lib-1.0-SNAPSHOT.jar
[INFO]
[INFO] --- install:3.1.2:install (default-install) @ my-lib ---
[INFO] Installing E:\my-lib\pom.xml to C:\Users\Aleksandra Zakharova\.m2\repository\sasha\example\my-lib\1.0-SNAPSHOT\my-lib-1.0-SNAPSHOT.pom
[INFO] Installing E:\my-lib\target\my-lib-1.0-SNAPSHOT.jar to C:\Users\Aleksandra Zakharova\.m2\repository\sasha\example\my-lib\1.0-SNAPSHOT\my-lib-1.0-SNAPSHOT.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.442 s
[INFO] Finished at: 2025-10-03T13:01:14+03:00
[INFO] ------------------------------------------------------------------------

Process finished with exit code 0
```

Опубликовалась в локальный репозиторий

1.0-SNAPSHOT

Файл | Главная | Поделиться | Вид

Этот компьютер › Acer (C:) › Пользователи › Aleksandra Zakharova › .m2 › repository › sasha › example › my-lib › 1.0-SNAPSHOT

Поиск в: 1.0-SNAP...

- Видео
- Документы
- Загрузки
- Изображения
- Музыка
- Объемные объекты
- Рабочий стол
- Acer (C:)
- Other (F:)

Имя	Дата изменения	Тип	Размер
_remote.repositories	03.10.2025 13:01	Файл "REPOSITOR...	1 КБ
maven-metadata-local.xml	03.10.2025 13:01	Файл "XML"	1 КБ
my-lib-1.0-SNAPSHOT.jar	03.10.2025 13:01	Executable Jar File	5 КБ
my-lib-1.0-SNAPSHOT.pom	03.10.2025 13:00	Файл "POM"	1 КБ

Элементов: 4

maven-metadata-local.xml

IThub

C:\Users\Aleksandra Zakharova\.m2\repository\sasha\example\my-lib\1.0-SNAPSHOT\maven-metadata-local.xm...

Файл Правка Поиск Вид Кодировки Синтаксисы Опции Инструменты Макросы Запуск Плагины Вкладки ? X

maven-metadata-local.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<metadata modelVersion="1.1.0">
  <groupId>sasha.example</groupId>
  <artifactId>my-lib</artifactId>
  <versioning>
    <lastUpdated>20251003100111</lastUpdated>
    <snapshot>
      <localCopy>true</localCopy>
    </snapshot>
    <snapshotVersions>
      <snapshotVersion>
        <extension>pom</extension>
        <value>1.0-SNAPSHOT</value>
        <updated>20251003100111</updated>
      </snapshotVersion>
      <snapshotVersion>
        <extension>jar</extension>
        <value>1.0-SNAPSHOT</value>
        <updated>20251003100111</updated>
      </snapshotVersion>
    </snapshotVersions>
  </versioning>
  <version>1.0-SNAPSHOT</version>
</metadata>

```

eXtensi length : 699 lines : 25 | Ln : 1 Col : 1 Sel : 0 | 0 | Unix (LF) | UTF-8 | INS

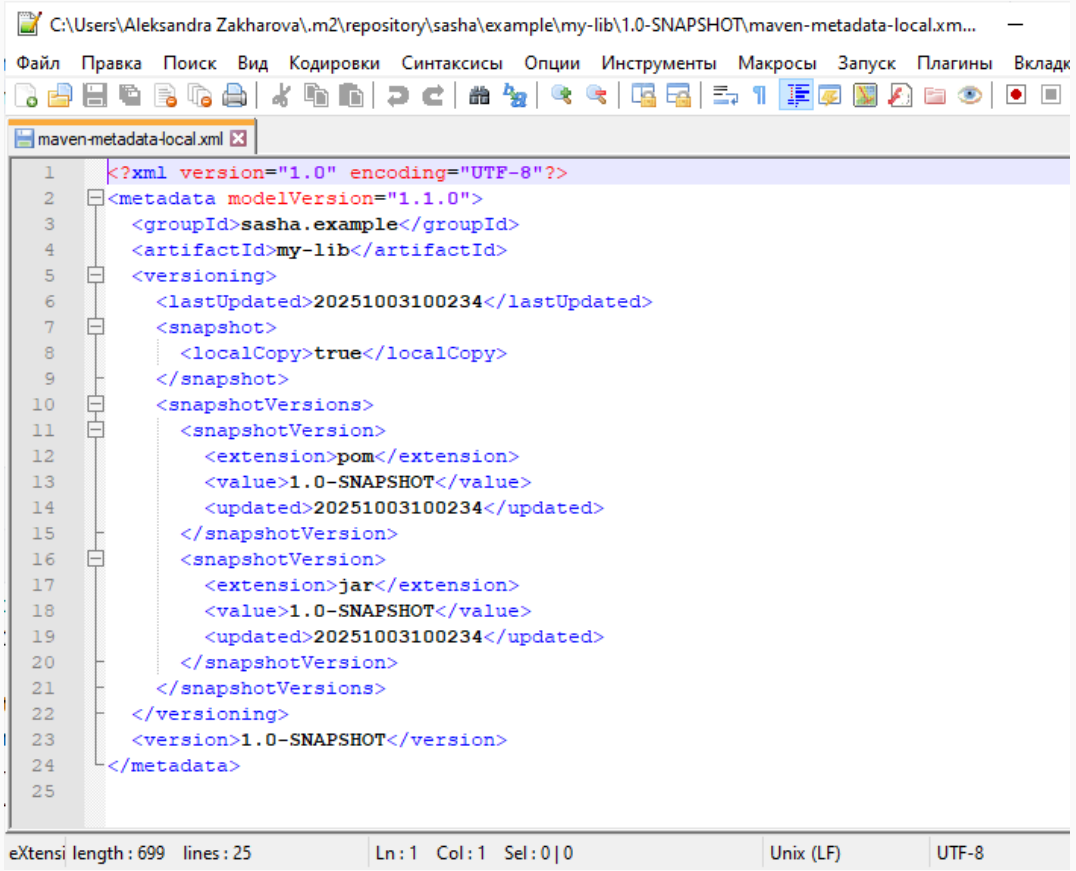
При каждом изменении кода и переопубликации артефакта (mvn install) в maven-metadata-local.xml изменится информация о создании

```java
public class StringUtils {

    public static String toLowerCase(String str) {
        return str.toLowerCase();
    }

    public static String toUpperCase(String str) {
        return str.toUpperCase();
    }

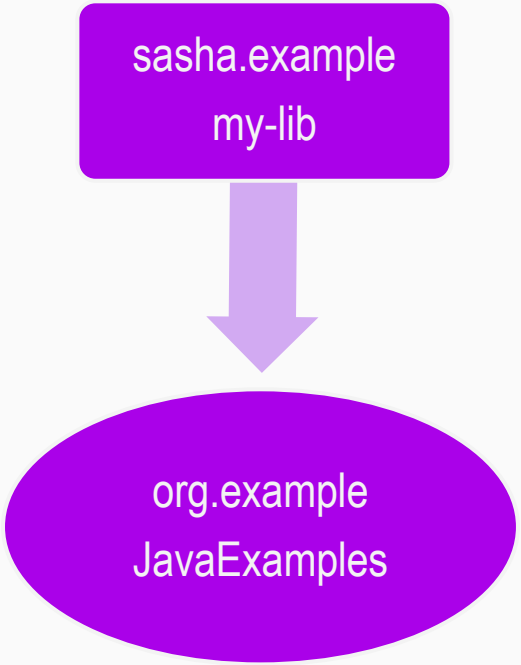
    public static String trim(String str) {
        return str.trim();
    }
}
```



попробуем использовать код из my-lib в другом проекте (JavaExamples):

IThub

```
<groupId>sasha.example</groupId>
 <artifactId>my-lib</artifactId>
 <version>1.0-SNAPSHOT</version>
```




```
<groupId>org.example</groupId>
<artifactId>JavaExamples</artifactId>
<version>2.2-SNAPSHOT</version>

……

<repositories>
   <repository>
      <id>local-libs</id>
      <name>Libs Local Repository</name>
      <url>file://C:\Users\Aleksandra Zakharova\.m2</url>
   </repository>
</repositories>

…..

<dependencies>
    ……
    <dependency>
      <groupId>sasha.example</groupId>
      <artifactId>my-lib</artifactId>
      <version>1.0-SNAPSHOT</version>
   </dependency>
</dependencies>
```

Из JavaExamples

IThub

```java
package org.example;

import sasha.example.LogUtils;

public class LibExample {

    public static void main(String[] args) {
        String debugMessage = LogUtils.getDebugMessage("message from JavaExamples");
        System.out.println(debugMessage);
    }
}
```



Внесем изменения в либу в метод *getDebugMessage*

IThub

```
package sasha.example;

public class LogUtils {

    public static String getDebugMessage(String message) {
        return "[DEBUG] " + message + " (new message!)";
    }

    public static String getInfoMessage(String message) {
        return "[INFO] " + message;
    }

    public static String getWarnMessage(String message) {
        return "[WARN] " + message;
    }
}
```

пересобираем артефакт (mvn install)

Возвращаемся в JavaExamples (нам **не надо** менять версию в pom.xml – подтянется свежий snapshot)

IThub

```
package org.example;

import sasha.example.LogUtils;

public class LibExample {

    public static void main(String[] args) {
        String debugMessage = LogUtils.getDebugMessage("message from JavaExamples");
        System.out.println(debugMessage);
    }
}
```

```
<dependency>
    <groupId>sasha.example</groupId>
    <artifactId>my-lib</artifactId>
    <version>1.0-SNAPSHOT</version>
</dependency>
```



IThub

Конфликт зависимостей (dependency conflict) — это ситуация, когда разные библиотеки в Java-проекте требуют разные версии одной и той же библиотеки.

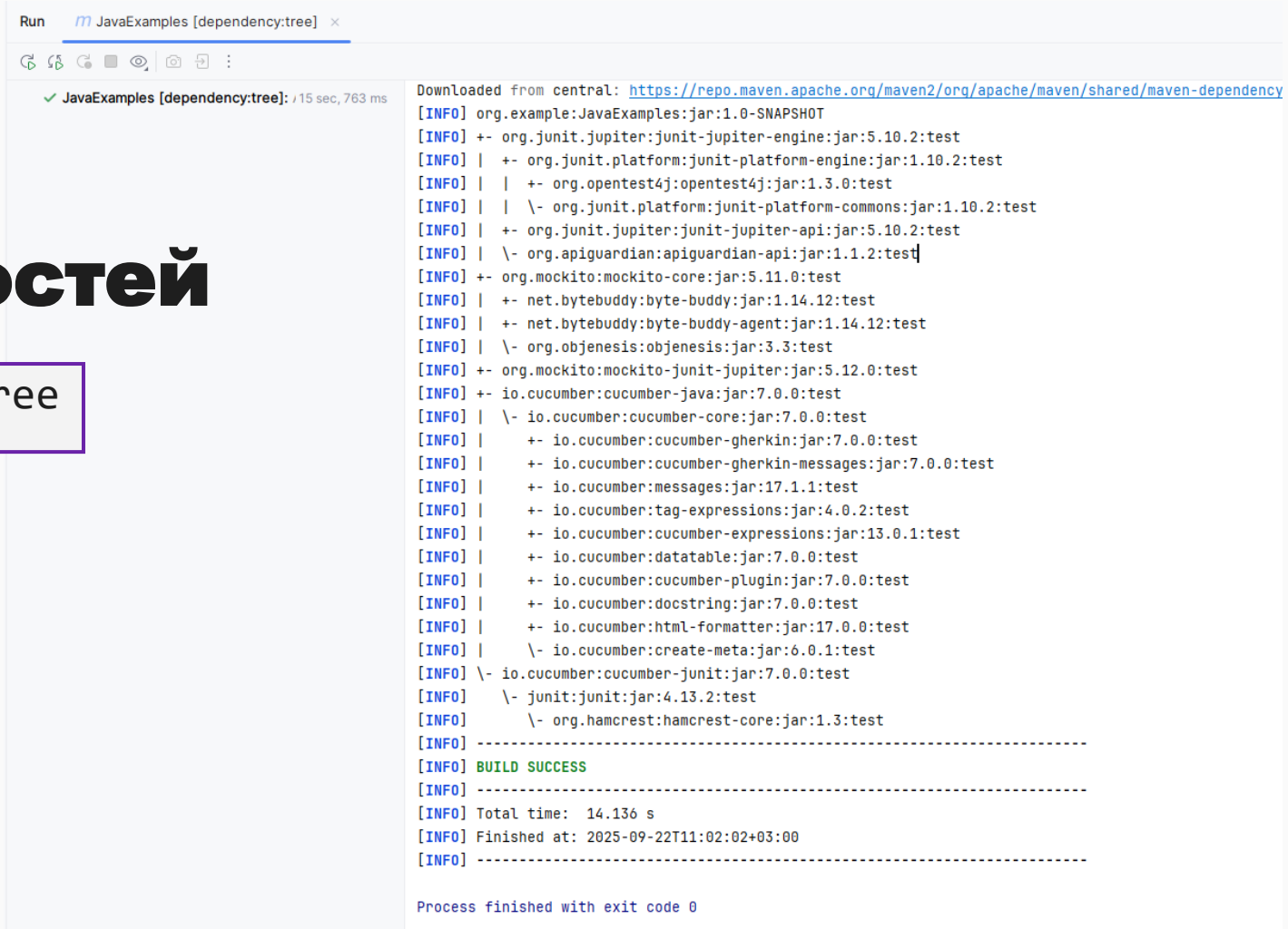
Зависимости бывают прямыми и транзитивными.

Прямые зависимости прописываются в pom.xml в секции <dependencies>

Транзитивная зависимость — это библиотека, которую явно не подключали, но она пришла вместе с другой библиотекой.

Дерево зависимостей

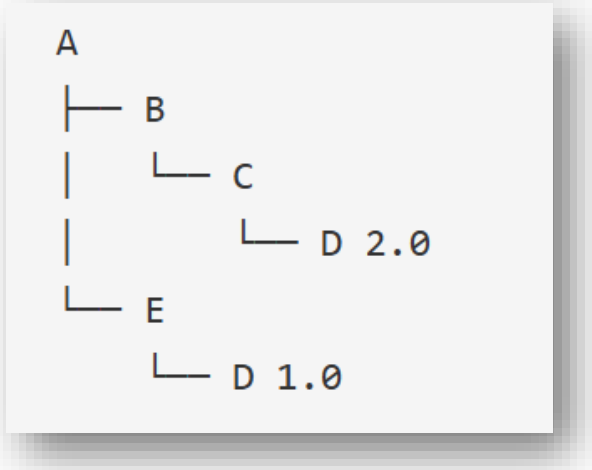
```
mvn dependency:tree
```



IThub

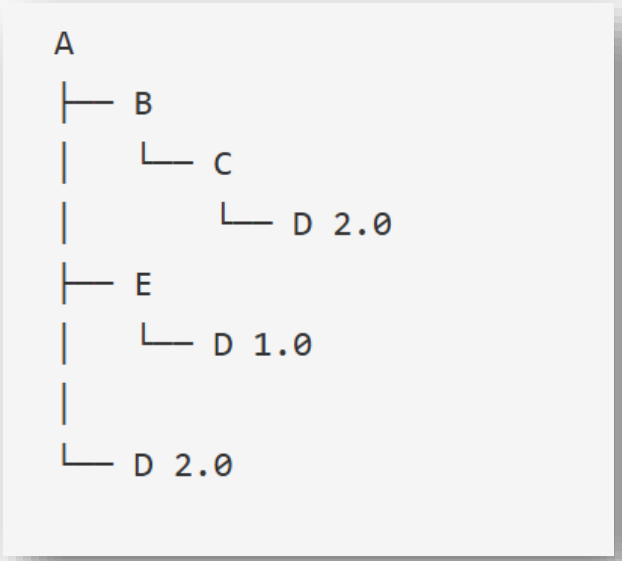
Maven выбирает «**ближайшее определение**» («**nearest definition**»). Это означает, что используемая версия будет наиболее близкой к вашему проекту в дереве зависимостей.

Например есть дерево зависимостей:



При построении A будет использоваться D 1.0, поскольку путь от A к D через E короче

Можно явно добавить зависимость к D 2.0 в A, чтобы принудительно использовать D 2.0



IThub

IThub

Как решить конфликт зависимостей?

Способ 1. Явно указать нужную версию в pom.xml

Способ 2. Использовать dependencyManagement

Способ 3. Исключить транзитивную зависимость через секцию в <exclusions> в самой <dependency>

Dependency management - позволяет авторам проекта напрямую указывать версии артефактов, которые будут использоваться, когда они встречаются в транзитивных зависимостях или в зависимостях, где версия не указана.

```
<dependencyManagement>
   <dependencies>
      <dependency>
         <groupId>junit</groupId>
         <artifactId>junit</artifactId>
         <version>4.13.2</version>
         <scope>test</scope>
      </dependency>
   </dependencies>
</dependencyManagement>
```

```
<dependencies>
   <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
   </dependency>
</dependencies>
```

Maven-плагины

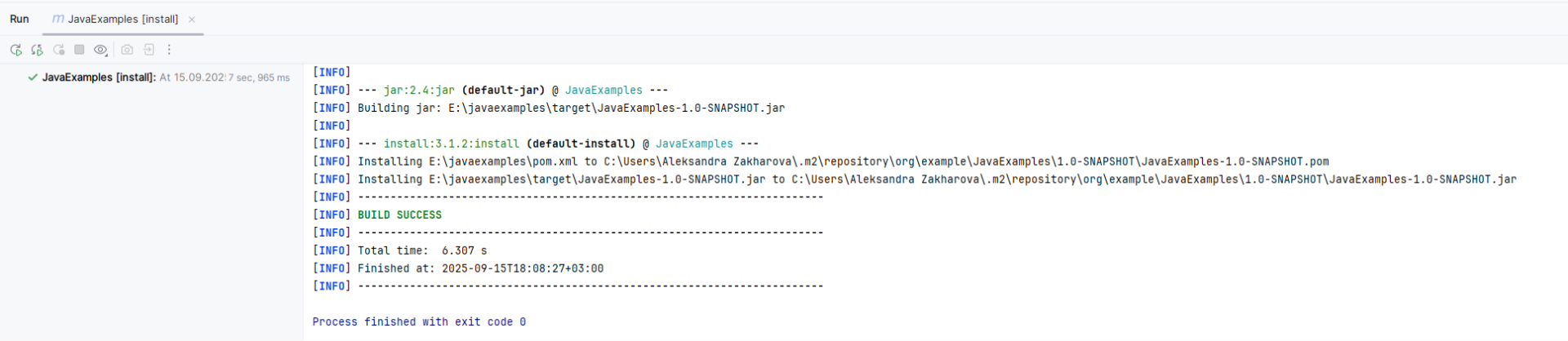
- Иногда во время какой-то фазы нужно выполнить дополнительный функционал. Для этого существуют Maven-плагины. Список официальных плагинов можно посмотреть на сайте Мавена.
- Чтобы добавить Мавен-плагин в проект, его описание, аналогично зависимостям, нужно поместить в pom.xml в тег <build> и <plugins>.

IThub

```
...
<groupId>org.example</groupId>
<artifactId>JavaExamples</artifactId>
<version>1.0-SNAPSHOT</version>

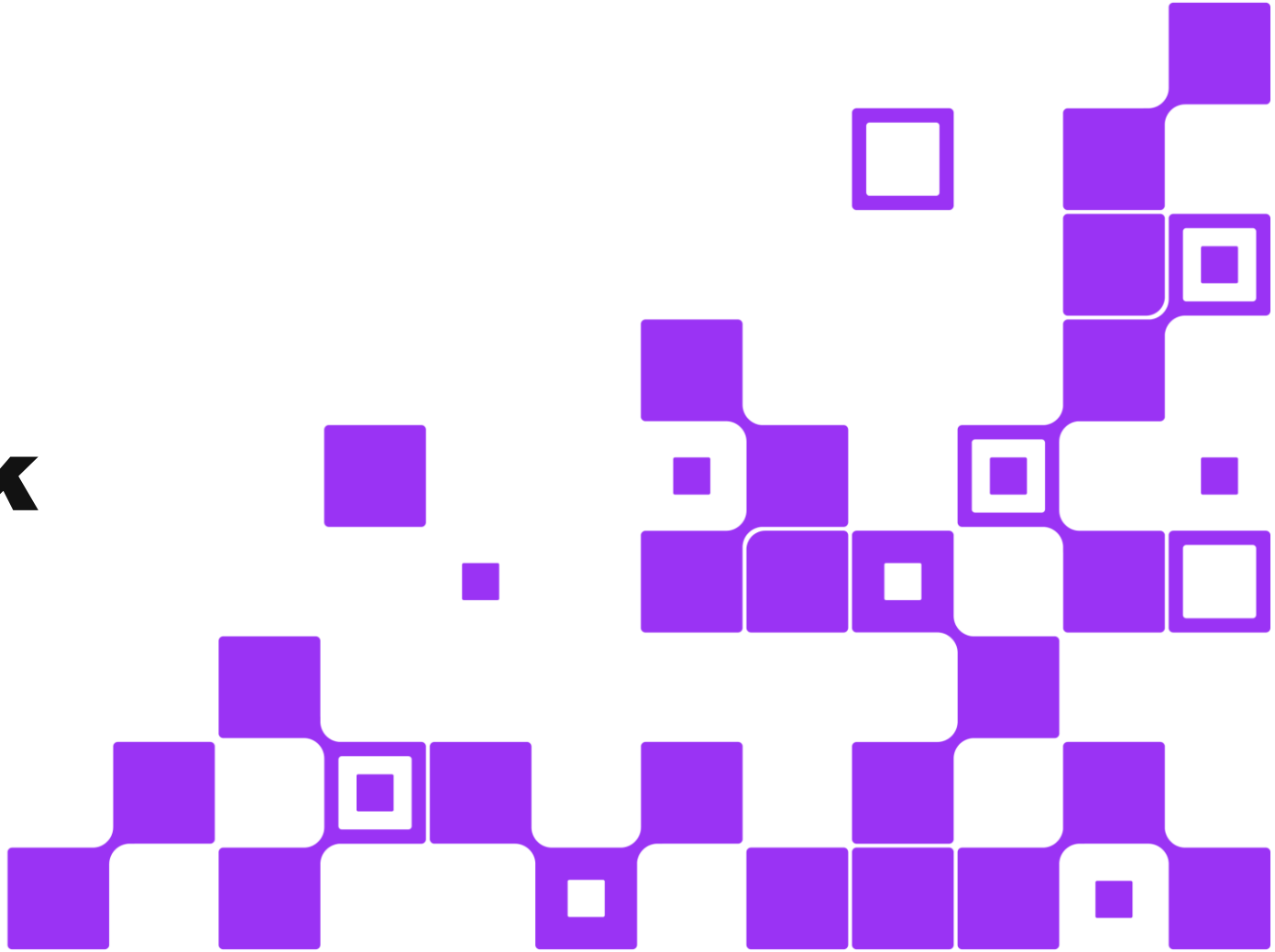
<properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-
8</project.build.sourceEncoding>
</properties>
```

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-jar-plugin</artifactId>
      <version>2.4</version>
      <configuration>
        <includes>
          <include>**/properties/*</include>
        </includes>
        <excludes>
          <exclude>**/*.txt</exclude>
        </excludes>
      </configuration>
    </plugin>
  </plugins>
</build>
```



IThub

Lombok



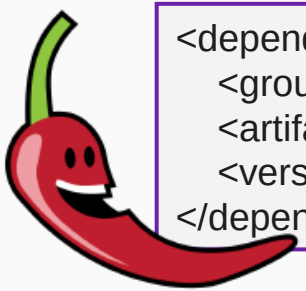
Project Lombok

IThub

Lombok — это Java-библиотека, которая автоматически интегрируется в проект и инструменты сборки, делая работу с Java более эффективной и приятной.

Больше не придется вручную писать:

- геттеры и сеттеры
- конструкторы
- toString
- equals и hashcode



```
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <version>1.18.36</version>
</dependency>
```

Аннотации Lombok

IThub

@Getter и **@Setter**	геттеры и сеттеры для поля. Могут использоваться как на уровне поля, так и на уровне класса.
@NoArgsConstructor	используется для создания конструктора без аргументов
@AllArgsConstructor	генерирует параметризованный конструктор, который принимает один параметр для каждого поля и инициализирует их с его помощью
@ToString	переопределяет метод toString() и создает для него реализацию по умолчанию. Она выводит имя класса и поля по порядку, разделяя их запятыми
@EqualsAndHashCode	применяется для переопределения методов equals() и hashCode()
@Data	объединяет аннотации @ToString, @Getter, @Setter, @EqualsAndHashCode и @RequiredArgsConstructor в одну

Класс без Lombok

```
public class Coffee {

    private String id;
    private String name;

    public Coffee() {
    }

    public Coffee(String id, String name) {
        this.id = id;
        this.name = name;
    }

    public String getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setId(String id) {
        this.id = id;
    }

    @Override
    public boolean equals(Object o) {
        if (o == null || getClass() != o.getClass()) return false;
        Coffee coffee = (Coffee) o;
        return Objects.equals(id, coffee.id) && Objects.equals(name, coffee.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(id, name);
    }
```

Класс с Lombok

```
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
@EqualsAndHashCode
public class Coffee {

    private String id;
    private String name;
}
```

или

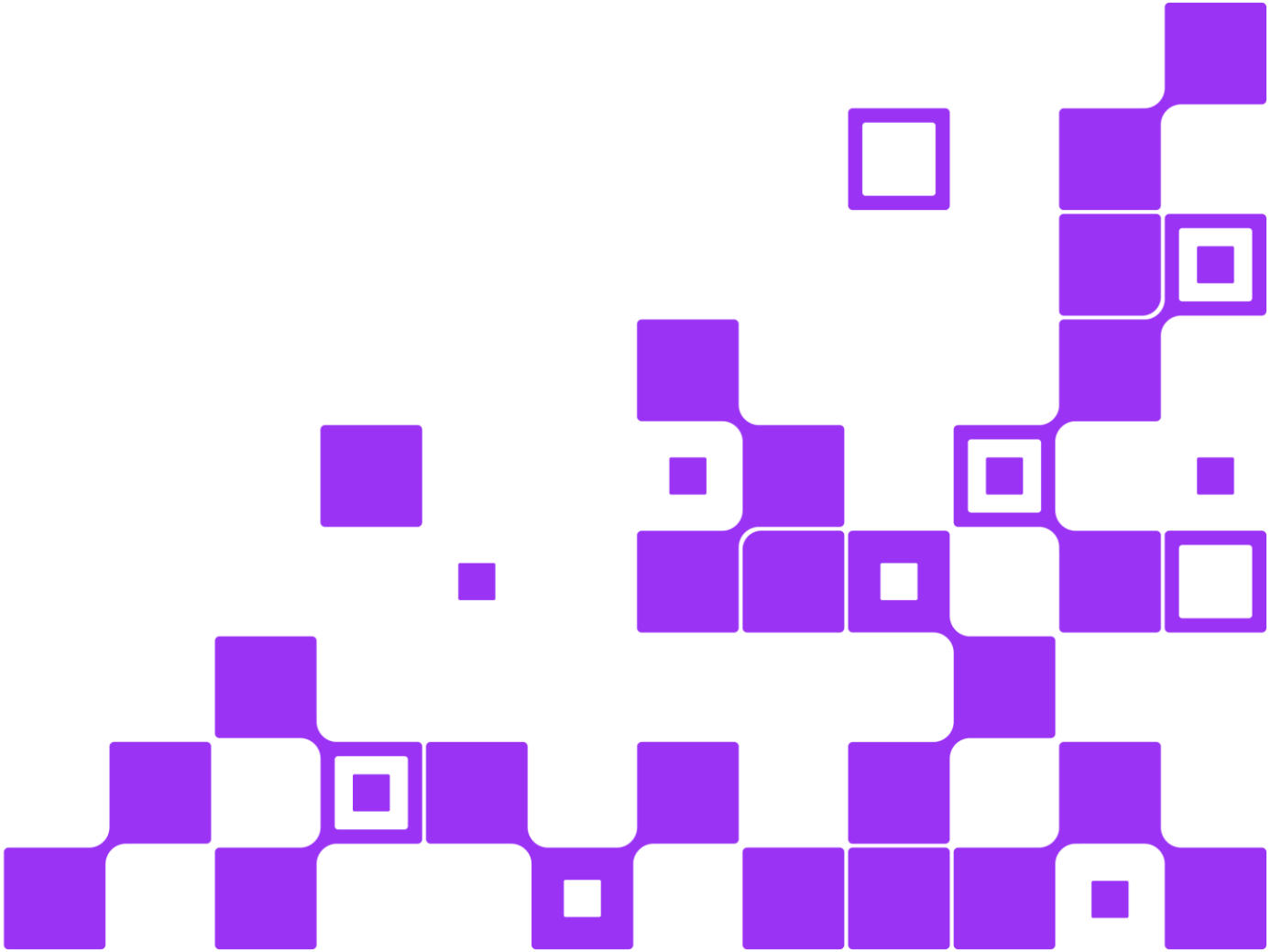
```
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Coffee {

    private String id;
    private String name;
}
```

IThub

IThub

JSON



JSON

IThub

JavaScript Object Notation (JSON) — текстовый формат обмена данными, основанный на JavaScript. Но при этом формат независим от JS и может использоваться в любом языке программирования.

JSON построен на двух структурах:

- коллекция пар «имя: значение»
- упорядоченный список значений

IThub

Объект

Имя объекта начинается с открывающей фигурной скобки { и заканчивается закрывающей фигурной скобкой }.
За каждым именем следует двоеточие (:),
а пары «имя: значение» разделяются запятой (,)

```
{
  "firstName": "Иван",
  "lastName": "Иванов",
  "address": {
      "streetAddress": "Московское ш., 101, кв.101",
      "city": "Нижний Новгород"
   },
  "phoneNumber": "812 123-1234 “
}
```

Значение

Значение может быть

- строкой в двойных кавычках
- числом,
- значением true, false или null
- объектом
- массивом.

Эти структуры могут быть вложенными.

```
{
   "firstName": "Иван",
   "lastName": "Иванов",
   "children" : null,
   "married": true,
   "age" : 27,
   "address": {
      "streetAddress": “Новая ул., 1, кв.1",
      "city": "Нижний Новгород"
    },
   "phoneNumbers": [
      "812 123-1234",
      "916 123-4567"
   ]
}
```

IThub

JSON Schema

JSON Schema - это декларативный язык для определения структуры и ограничений для данных JSON.

Описание структурированных данных

позволяет разработчикам описывать структуру, ограничения и типы данных существующих данных

Определение и применение правил

соблюдение ограничений схемы JSON упрощает обмен структурированными данными между приложениями, поскольку сохраняется единообразие шаблонов.

Понятная документация

JSON Schema поддерживает создание документации, удобной для чтения.

Автоматизированное тестирование

валидация обеспечивает автоматизированное тестирование, гарантируя, что данные последовательно соответствуют заданным правилам и ограничениям.

JSON Schema позволяет описать:

IThub

- Формат данных (числа, строки, объекты, массивы)
- Допустимые значения (минимумы, максимумы)
- Обязательные поля и необязательные

json схема:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Product",
  "description": "A product from the catalog",
  "type": "object",
  "properties": {
    "id": {
      "description": "The unique identifier for a product",
      "type": "integer"
    },
    "name": {
      "description": "Name of the product",
      "type": "string"
    },
    "price": {
      "type": "number",
      "exclusiveMinimum": 0
    }
  },
  "required": ["id", "name", "price"]
}
```

json файл:

```
{
  "id": 1,
  "name": "Shampoo",
  "price": 0
}
```

IThub

Type

Указывает **тип данных**, который будет использоваться схемой. Это ключевое слово не является обязательным, и значением ключевого слова может быть строка, представляющая допустимый тип данных, или массив строк, представляющих допустимые типы данных.

```
{"type": ["number", "string", "null", "boolean"]}

{"type": "array"}
```

"type": "string"

Выполняется проверка на соответствие символьного выражения заданным параметрам:

- **minLength** (минимальная длина)
- **maxLength** (максимальная длина)
- **pattern** (используется для ограничения строки определенным регулярным выражением)
- **contentEncoding**
- **contentMediaType**

```
{
    "type": "string",
    "minLength": 3,
    "maxLength": 10,
    "pattern": "^test\\/[a-z-]+$",
    "contentEncoding": "base64",
    "contentMediaType": "application/json"
}
```

"type": "number" | "type": "integer"

Выполняется проверка на соответствие числового выражения заданным параметрам:

- **minimum** (нижняя граница)
- **exclusiveMinimum** (нижняя граница числового экземпляра, где экземпляр должен быть строго больше указанного значения)
- **maximum** (верхняя граница)
- **exclusiveMaximum** (верхняя граница числового экземпляра, где экземпляр должен быть строго меньше указанного значения)
- **multipleOf** (ограничивает до кратного заданному числу)

если exclusiveMinimum = false, x ≥ minimum

если exclusiveMinimum = true, x > minimum

```
{
    "type": "number",
    "minimum": 1.5,
    "exclusiveMinimum": true,
    "maximum": 12.3,
    "exclusiveMaximum": true,
    "multipleOf": 0.5
}
```

"type": "object"

Объект проверяется как по наименованию ключа, так и по ограничениям на значение.

Установка для схемы **additionalProperties** значения false означает, что никакие дополнительные свойства не будут разрешены.

Можно предоставить список обязательных свойств, используя ключевое слово **required**.

```
{
    "type": "object",
    "properties": {
      "name": { "type": "string" },
      "email": { "type": "string" },
      "address": { "type": "string" },
      "telephone": { "type": "string" }
    },
    "additionalProperties": false,
    "required": ["name", "email"]
}
```

"type": "array"

Массив проверяется как по наименованию ключа, так и по ограничениям на значение:

- **minItems** (минимальная длина массива)
- **maxItems** (максимальная длина массива)
- **uniqueItems** (если true - каждый из элементов массива должен быть уникален)
- **contains** (только по одному или нескольким элементам в массиве)
- **items** (для проверки всех элементов в массиве)
- **additionalItems** (запрет дополнительных элементов если = false)
- **enum** - допустимые значения
- **enumNames** - названия для отображения

```
"employees": {
 "description": "Сотрудники",
 "type": "array",
 "minItems" : 1,
 "maxItems" : 30000,
 "uniqueItems": true,
 "items": {
    "type": "object",
    "properties": {
          "name": {
               "type": "string",
               "enum": ["employee"],
               "enumNames": ["Сотрудник"]
          },
          "enabled": {
                "type": "boolean",
                "default": true
          }
 },
 "additionalItems": false
 }
}
```

Наложение условий на проверки

not - не должно быть допустимым для данной схемы:

```
{
    "not": {
      "type": "string"
    }
}
```

42

{ "key": "value" }

I am a string

42

{ "key": "value" }

I am a string

allOf - чтобы все элементы были действительными для всех заданных подсхем:

```
{
  "allOf": [
    { "type": "string" },
    { "maxLength": 5 }
  ]
}
```

short

too long

short

too long

Наложение условий на проверки

anyOf - должно быть допустимо для *любой* из подсхем:

```
{
  "anyOf": [
    { "type": "string", "maxLength": 5 },
    { "type": "number", "minimum": 0 }
  ]
}
```

short
too long
12
-5

short
too long
12
-5

oneOf - должно быть допустимо только для *одной* из подсхем:

```
{
  "oneOf": [
    { "type": "number", "multipleOf": 5 },
    { "type": "number", "multipleOf": 3 }
  ]
}
```

10
2
9
15

10
2
9
15

Пример

```
{
  "id": 1,
  "name": "Shampoo",
  "price": 0
}
```

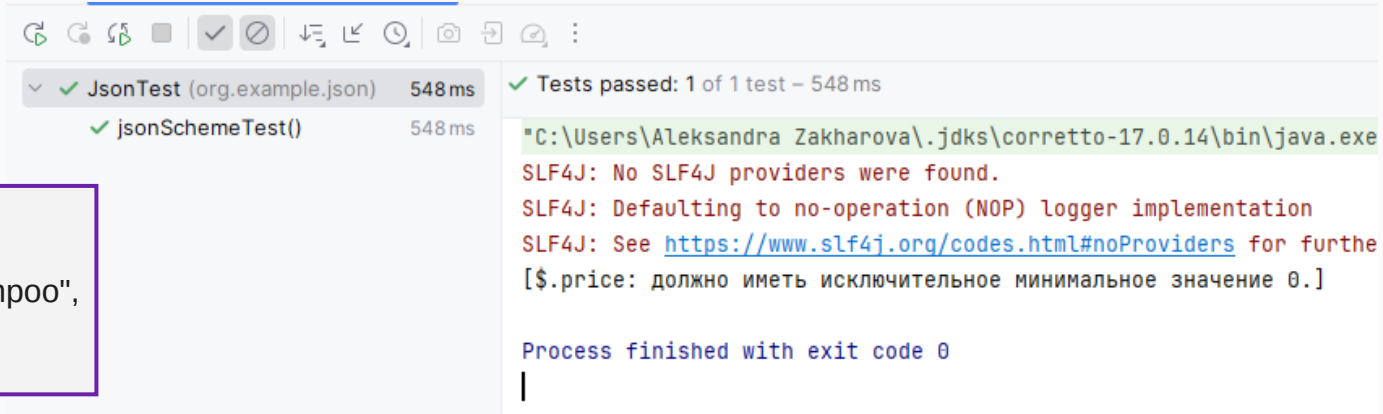
```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Product",
  "description": "A product from the catalog",
  "type": "object",
  "properties": {
    "id": {
      "description": "The unique identifier for a product",
      "type": "integer"
    },
    "name": {
      "description": "Name of the product",
      "type": "string"
    },
    "price": {
      "type": "number",
      "exclusiveMinimum": 0
    }
  },
  "required": ["id", "name", "price"]
}
```

```java
@Test
public void jsonSchemeTest() throws Exception {
    ObjectMapper mapper = new ObjectMapper();

    FileInputStream jsonExample = new FileInputStream(new File("src/test/resources/json/product_invalid.json"));
    FileInputStream schemaExample = new FileInputStream(new File("src/test/resources/json/schema.json"));

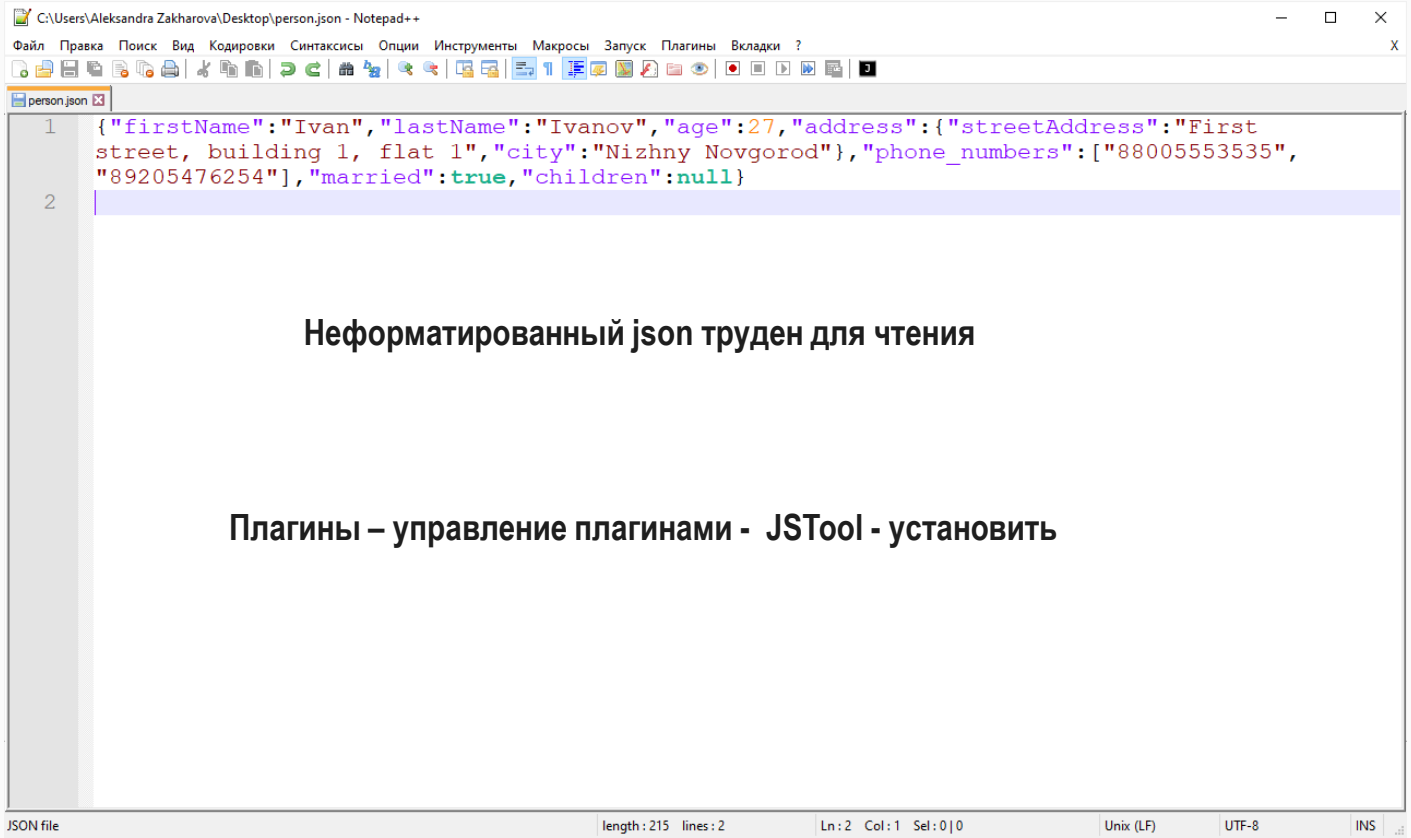
    JsonSchemaFactory factory = JsonSchemaFactory.getInstance(SpecVersion.VersionFlag.V4);
    JsonSchema jsonSchema = factory.getSchema(schemaExample);
    JsonNode jsonNode = mapper.readTree(jsonExample);
    Set<ValidationMessage> errors = jsonSchema.validate(jsonNode);

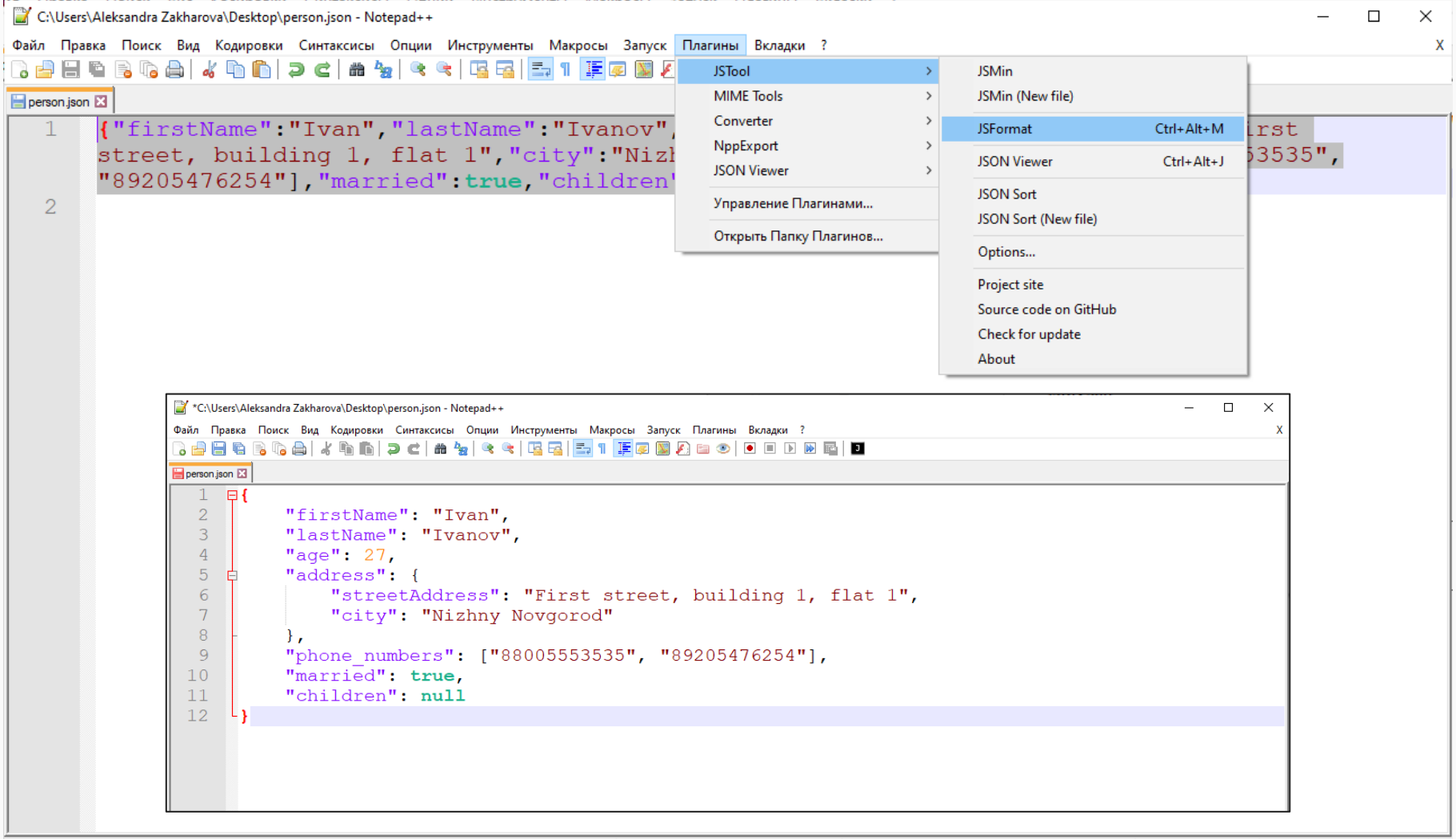
    Assertions.assertFalse(errors.isEmpty());
    System.out.println(errors);
}
```




```
{
  "id": 1,
  "name": "Shampoo",
  "price": 0
}
```

Json в notepad++




C:\Users\Aleksandra Zakharova\Desktop\person.json - Notepad++
Файл Правка Поиск Вид Кодировки Синтаксисы Опции Инструменты Макросы Запуск Плагины Вкладки ?
person.json
{"firstName":"Ivan","lastName":"Ivanov",
street, building 1, flat 1","city":"Nizh
"89205476254"],"married":true,"children"
JSTool
MIME Tools
Converter
NppExport
JSON Viewer
Управление Плагинами...
Открыть Папку Плагинов...
JSMin
JSMin (New file)
JSFormat Ctrl+Alt+M
JSON Viewer Ctrl+Alt+J
JSON Sort
JSON Sort (New file)
Options...
Project site
Source code on GitHub
Check for update
About
*C:\Users\Aleksandra Zakharova\Desktop\person.json - Notepad++
Файл Правка Поиск Вид Кодировки Синтаксисы Опции Инструменты Макросы Запуск Плагины Вкладки ?
person.json
{
"firstName": "Ivan",
"lastName": "Ivanov",
"age": 27,
"address": {
"streetAddress": "First street, building 1, flat 1",
"city": "Nizhny Novgorod"
},
"phone_numbers": ["88005553535", "89205476254"],
"married": true,
"children": null
}
JSON file
length : 215 lines : 2
Ln : 1 Col : 1 Sel : 215 | 2
Unix (LF)
UTF-8
INS

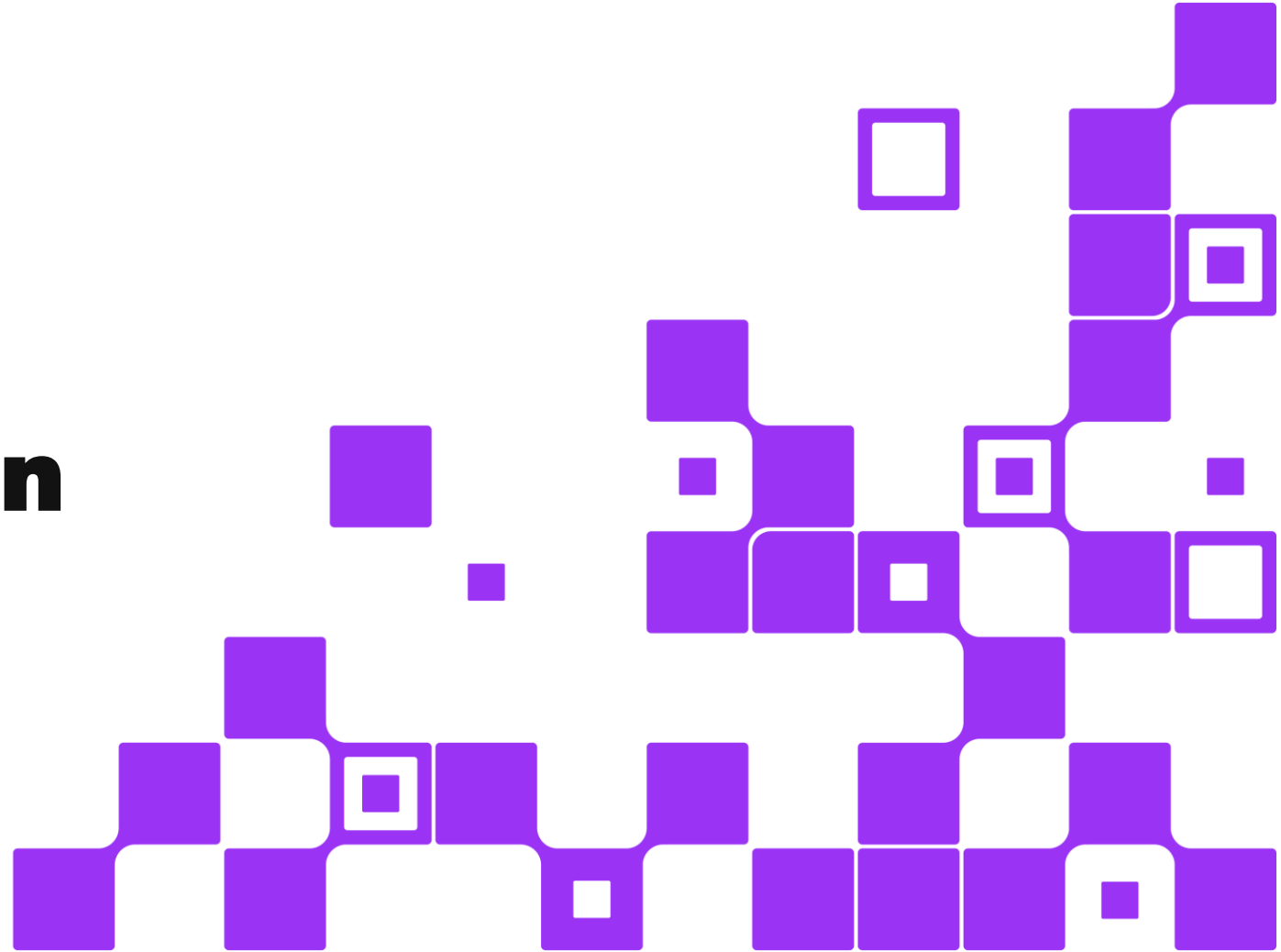
Библиотеки для работы с Json для java

IThub

- GSON
- Simple Json
- Jackson
- JsonPath
- LoganSquare (для Android)
- JSONUtil

IThub

Jackson



IThub

Jackson - популярная библиотека для сериализации/ десериализации Java-объектов в различные текстовые форматы

основной функционал для работы с форматом JSON — класс ObjectMapper

интегрирован в Spring Framework

простая привязка данных - преобразует JSON в Java Maps, Lists, Strings, Numbers, Booleans, null объекты и обратно

полная привязка данных - преобразует JSON в любой класс Java и из него

```
<dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.20.0</version>
</dependency>
```

POJO объекты

IThub

POJO (Plain Old Java Object) — это обычный Java-объект, который не зависит от специальных фреймворков, библиотек или контейнеров.

Проще говоря, это «простой класс Java» с полями и методами.

- содержит только данные и простую логику
- не наследуется от какого-то обязательного базового класса
- не требует реализации специальных интерфейсов
- не зависит от Spring, Hibernate или других библиотек

Создаем json

IThub

```
{
  "firstName": "Ivan",
  "lastName": "Ivanov",
  "children" : null,
  "married": true,
  "age" : 27,
  "address": {
    "streetAddress": "First street, building 1, flat 1",
    "city": "Nizhny Novgorod"
  },
  "phoneNumbers": [
    "88005553535",
    "89205476254"
  ]
}
```

```
@Getter
@Setter
@ToString
@AllArgsConstructor
@NoArgsConstructor
public class Person {
    private String firstName;
    private String lastName;
    private List<String> children;
    private boolean married;
    private int age;
    private Address address;
    private List<String> phoneNumbers;
}
```

POJO объекты

```
@Getter
@Setter
@ToString
@AllArgsConstructor
@NoArgsConstructor
public class Address {

    private String streetAddress;
    private String city;
}
```

Создаем json

IThub

```
@Test
public void jacksonCreateTest() throws Exception {
    ObjectMapper mapper = new ObjectMapper();

    Address address = new Address("First street, building 1, flat 1", "Nizhny Novgorod");
    List<String> phoneNumbers = new ArrayList<>() {{
        add("88005553535");
        add("89205476254");
    }};
    Person person = new Person("Ivan", "Ivanov", null, true, 27, address, phoneNumbers);

    String personJson = mapper.writeValueAsString(person);
    System.out.println(personJson);
}
```

Парсим json

IThub

```
{
  "firstName": "Ivan",
  "lastName": "Ivanov",
  "children" : null,
  "married": true,
  "age" : 27,
  "address": {
    "streetAddress": "First street, building 1, flat 1",
    "city": "Nizhny Novgorod"
  },
  "phoneNumbers": [
    "88005553535",
    "89205476254"
  ]
}
```

```
@Getter
@Setter
@ToString
@AllArgsConstructor
@NoArgsConstructor
public class Person {
    private String firstName;
    private String lastName;
    private List<String> children;
    private boolean married;
    private int age;
    private Address address;
    private List<String> phoneNumbers;
}
```

```
@Getter
@Setter
@ToString
@AllArgsConstructor
@NoArgsConstructor
public class Address {

    private String streetAddress;
    private String city;
}
```

Парсим json

IThub

```
@Test
public void jacksonParseTest() throws Exception {
    String personJson = getTextFromFile(personJsonFilePath);
    ObjectMapper mapper = new ObjectMapper();
    Person person = mapper.readValue(personJson, Person.class);

    Assertions.assertEquals("Ivan", person.getFirstName());
    Assertions.assertEquals("Ivanov", person.getLastName());
    Assertions.assertNull(person.getChildren());
    Assertions.assertTrue(person.isMarried());
    Assertions.assertEquals(27, person.getAge());

    Assertions.assertEquals("First street, building 1, flat 1",
        person.getAddress().getStreetAddress());
    Assertions.assertEquals("Nizhny Novgorod",
        person.getAddress().getCity());

    Assertions.assertEquals("88005553535", person.getPhoneNumbers().get(0));
    Assertions.assertEquals("89205476254", person.getPhoneNumbers().get(1));
}
```

```
{
  "firstName": "Ivan",
  "lastName": "Ivanov",
  "children" : null,
  "married": true,
  "age" : 27,
  "address": {
    "streetAddress": "First street, building 1, flat 1",
    "city": "Nizhny Novgorod"
  },
  "phoneNumbers": [
    "88005553535",
    "89205476254"
  ],
  "carDriver": true
}
```

```
@Getter
@Setter
@ToString
@AllArgsConstructor
@NoArgsConstructor
public class Person {
    private String firstName;
    private String lastName;
    private List<String> children;
    private boolean married;
    private int age;
    private Address address;
    private List<String> phoneNumbers;
}
```

IThub

! **POJO объект всегда должен соответствовать структуре json-а**



Jackson аннотации

IThub

@JsonProperty	используется, чтобы указать имя свойства в JSON (ключ)
@JsonGetter	является альтернативой аннотации @JsonProperty, которая отмечает метод как метод-получатель
@JsonPropertyOrder	для указания порядка свойств при сериализации

```java
@Setter
@AllArgsConstructor
@NoArgsConstructor
@ToString
public class Person {
    private String firstName;
    private String lastName;
    private List<String> children;
    private boolean married;
    private int age;
    private Address address;
    private List<String> phone_numbers;

    @JsonGetter("firstName")
    public String getFirstName() {return firstName; }

    @JsonGetter("lastName")
    public String getLastName() { return lastName; }

    @JsonGetter("children")
    public List<String> getChildren() { return children; }

    @JsonGetter("married")
    public boolean isMarried() { return married; }

    @JsonGetter("age")
    public int getAge() { return age; }

    @JsonGetter("address")
    public Address getAddress() { return address; }

    @JsonGetter("phoneNumbers")
    public List<String> getPhone_numbers() { return phone_numbers; }
}
```

IThub

```java
ObjectMapper mapper = new ObjectMapper();

Address address = new Address("First street, building 1, flat 1", "Nizhny Novgorod");
List<String> phoneNumbers = new ArrayList<>() {{
    add("88005553535");
    add("89205476254");
}};
Person person = new Person("Ivan", "Ivanov", null, true, 27, address, phoneNumbers);

String personJson = mapper.writeValueAsString(person);
System.out.println(personJson);
```

```json
{
  "firstName": "Ivan",
  "lastName": "Ivanov",
  "children" : null,
  "married": true,
  "age" : 27,
  "address": {
    "streetAddress": "First street, building 1, flat 1",
    "city": "Nizhny Novgorod"
  },
  "phoneNumbers": [
    "88005553535",
    "89205476254"
  ]
}
```

```java
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
@ToString

@JsonPropertyOrder({ "firstName", "lastName", "age", "address", "phone_numbers", "married", "children" })
public class Person {
    private String firstName;
    private String lastName;
    private List<String> children;
    private boolean married;
    private int age;
    private Address address;
    private List<String> phone_numbers;

    @JsonGetter("phoneNumbers")
    public List<String> getPhone_numbers() {
        return phone_numbers; }
}
```

```
{
  "firstName": "Ivan",
  "lastName": "Ivanov",
  "age" : 27,
  "address": {
    "streetAddress": "First street, building 1, flat 1",
    "city": "Nizhny Novgorod"
  },
  "phoneNumbers": [
    "88005553535",
    "89205476254"
  ],
  "married": true,
  "children" : null
}
```

@JsonPropertyOrder

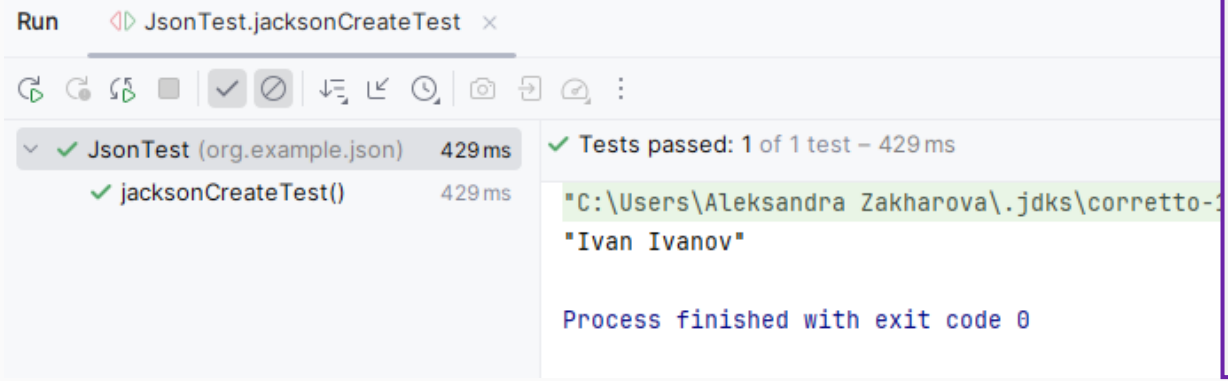
для указания порядка свойств при сериализации

IThub

IThub

@JsonValue

указывает единственный метод, который библиотека будет использовать для сериализации всего экземпляра.




```java
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
@ToString
public class Person {
    private String firstName;
    private String lastName;
    private List<String> children;
    private boolean married;
    private int age;
    private Address address;
    private List<String> phone_numbers;

    @JsonValue
    public String toFullName() {
        return firstName + " " + lastName;
    }
}
```

Jackson аннотации

IThub

@JsonCreator	Для настройки конструктора/фабрики, используемых при десериализации (когда нам нужно десериализовать JSON, который не совсем соответствует целевой сущности, которую нужно получить)
@JsonSetter	Альтернатива @JsonProperty, которая отмечает метод как метод-сеттер (когда целевой класс сущности не совсем соответствует json)
@JsonAlias	Определяет одно или несколько альтернативных имен для свойства во время десериализации

```java
@Getter
@Setter
@ToString
public class Person {
    private String firstName;
    private String lastName;
    private List<String> children;
    private boolean married;
    private int age;
    private Address address;
    private List<String> phone_numbers;

    @JsonCreator
    public Person(@JsonProperty("firstName") String firstName,
                  @JsonProperty("lastName") String lastName,
                  @JsonProperty("children") List<String> children,
                  @JsonProperty("married") boolean married,
                  @JsonProperty("age") int age,
                  @JsonProperty("address") Address address,
                  @JsonProperty("phoneNumbers") List<String>
phone_numbers) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.children = children;
        this.married = married;
        this.age = age;
        this.address = address;
        this.phone_numbers = phone_numbers;
    }
}
```

IThub

```java
String personJson = getTextFromFile(personJsonFilePath);
ObjectMapper mapper = new ObjectMapper();
Person person = mapper.readValue(personJson, Person.class);
```

```json
{
  "firstName": "Ivan",
  "lastName": "Ivanov",
  "children" : null,
  "married": true,
  "age" : 27,
  "address": {
    "streetAddress": "First street, building 1, flat 1",
    "city": "Nizhny Novgorod"
  },
  "phoneNumbers": [
    "88005553535",
    "89205476254"
  ]
}
```

```java
@Getter
@ToString
public class Person {
    private String firstName;
    private String lastName;
    private List<String> children;
    private boolean married;
    private int age;
    private Address address;
    private List<String> phone_numbers;

    @JsonSetter("firstName")
    public void setFirstName(String firstName) { this.firstName = firstName;
}

    @JsonSetter("lastName")
    public void setLastName(String lastName) { this.lastName = lastName;
}

    @JsonSetter("children")
    public void setChildren(List<String> children) { this.children = children; }

    @JsonSetter("married")
    public void setMarried(boolean married) { this.married = married; }

    @JsonSetter("age")
    public void setAge(int age) { this.age = age; }

    @JsonSetter("address")
    public void setAddress(Address address) { this.address = address; }

    @JsonSetter("phoneNumbers")
    public void setPhone_numbers(List<String> phone_numbers) {
        this.phone_numbers = phone_numbers;
    }
}
```

IThub

@JsonSetter

Это полезно, когда нужно прочитать некоторые данные JSON, но целевой класс сущности не совсем соответствует этим данным , и поэтому нам нужно настроить процесс, чтобы он соответствовал.

```json
{
  "firstName": "Ivan",
  "lastName": "Ivanov",
  "children" : null,
  "married": true,
  "age" : 27,
  "address": {
    "streetAddress": "First street, building 1, flat 1",
    "city": "Nizhny Novgorod"
  },
  "phoneNumbers": [
    "88005553535",
    "89205476254"
  ]
}
```

@JsonAlias

определяет одно или несколько альтернативных имен для свойства во время десериализации

```java
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
@ToString
public class Person {
    private String firstName;
    private String lastName;
    private List<String> children;
    private boolean married;
    private int age;
    private Address address;
    @JsonAlias({ "phone_numbers", "phoneNumbers" })
    private List<String> phone_numbers;
}
```

```json
{
  "firstName": "Ivan",
  "lastName": "Ivanov",
  "children" : null,
  "married": true,
  "age" : 27,
  "address": {
    "streetAddress": "First street, building 1, flat 1",
    "city": "Nizhny Novgorod"
  },
  "phoneNumbers": [
    "88005553535",
    "89205476254"
  ]
}
```

@JsonFormat

настройка формата даты

```java
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
@ToString
public class Person {
    private String firstName;
    private String lastName;
    private List<String> children;
    private boolean married;
    private int age;
    @JsonFormat( pattern="yyyy-MM-dd" )
    private LocalDate birthday;
    private Address address;
    @JsonAlias({ "phone_numbers", "phoneNumbers" })
    private List<String> phone_numbers;
}
```

```json
{
  "firstName": "Ivan",
  "lastName": "Ivanov",
  "children" : null,
  "married": true,
  "age" : 27,
  "birthday" : "1999-08-28",
  "address": {
    "streetAddress": "First street, building 1, flat 1",
    "city": "Nizhny Novgorod"
  },
  "phoneNumbers": [
    "88005553535",
    "89205476254"
  ]
}
```

Jackson аннотации включения\выключения свойств

IThub

@JsonIgnoreProperties	аннотация **уровня класса**, которая отмечает свойство или список свойств, которые Jackson будет игнорировать
@JsonIgnore	используется для обозначения свойства, которое следует игнорировать на **уровне поля**
@JsonInclude	для исключения свойств с пустыми/нулевыми/значениями по умолчанию

@JsonIgnoreProperties

IThub

Аннотация **уровня класса**, которая отмечает свойство или список свойств, которые Jackson будет игнорировать

```
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
@ToString

@JsonIgnoreProperties({ "children", "married" })
public class Person {
    private String firstName;
    private String lastName;
    private List<String> children;
    private boolean married;
    private int age;
    private Address address;
    private List<String> phone_numbers;
}
```

```
{
  "firstName": "Ivan",
  "lastName": "Ivanov",
  "children" : null,
  "married": true,
  "age" : 27,
  "address": {
    "streetAddress": "First street, building 1, flat 1",
    "city": "Nizhny Novgorod"
  },
  "phoneNumbers": [
    "88005553535",
    "89205476254"
  ]
}
```

Несмотря на то, что поля в json файле заполнены, после дессериализации у объекта они будут пустые (null для списка, false по умолчанию для boolean)

```
Assertions.assertNull(person.getChildren());
Assertions.assertFalse(person.isMarried());
```

```java
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
@ToString
public class Person {
    private String firstName;
    private String lastName;
    @JsonIgnore
    private List<String> children;
    @JsonIgnore
    private boolean married;
    private int age;
    private Address address;
    private List<String> phone_numbers;
}
```

```java
Address address = new Address("First street, building 1, flat 1", "Nizhny Novgorod");
List<String> phoneNumbers = new ArrayList<>() {{
    add("88005553535");
    add("89205476254");
}};
Person person = new Person("Ivan", "Ivanov", null, true, 27, address, phoneNumbers);
```

@JsonIgnore

используется для обозначения свойства, которое следует игнорировать **на уровне поля**

После сериализации в json файле нет информации о полях married и children

```json
{
  "firstName": "Ivan",
  "lastName": "Ivanov",
  "age" : 27,
  "address": {
    "streetAddress": "First street, building 1, flat 1",
    "city": "Nizhny Novgorod"
  },
  "phoneNumbers": [
    "88005553535",
    "89205476254"
  ]
}
```

IThub

@JsonInclude

для исключения свойств с пустыми/нулевыми/значениями по умолчанию

IThub

```
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
@ToString

@JsonInclude(JsonInclude.Include.NON_NULL)
public class Person {
    private String firstName;
    private String lastName;
    private List<String> children;
    private boolean married;
    private int age;
    private Address address;
    private List<String> phone_numbers;
}
```

```
Address address = new Address("First street, building 1, flat 1", "Nizhny Novgorod");
List<String> phoneNumbers = new ArrayList<>() {{
    add("88005553535");
    add("89205476254");
}};
Person person = new Person("Ivan", "Ivanov", null, true, 27, address, phoneNumbers);
```

После сериализации в json файле нет информации о поле children

```
{
  "firstName": "Ivan",
  "lastName": "Ivanov",
  "married": true,
  "age" : 27,
  "address": {
    "streetAddress": "First street, building 1, flat 1",
    "city": "Nizhny Novgorod"
  },
  "phoneNumbers": [
    "88005553535",
    "89205476254"
  ]
}
```

Jackson JsonNode

IThub

```java
@Test
public void jacksonJsonNodeTest() throws Exception {
    String personJson = getTextFromFile(personJsonFilePath);
    ObjectMapper mapper = new ObjectMapper();
    JsonNode actualObj = mapper.readTree(personJson);

    Assertions.assertEquals("Ivan", actualObj.get("firstName").asText());
    Assertions.assertEquals("Ivanov", actualObj.get("lastName").asText());
    Assertions.assertTrue(actualObj.get("children").isEmpty());
    Assertions.assertTrue(actualObj.get("married").asBoolean());
    Assertions.assertEquals(27, actualObj.get("age").asInt());

    JsonNode address = actualObj.get("address");
    Assertions.assertEquals("First street, building 1, flat 1", address.get("streetAddress").asText());
    Assertions.assertEquals("Nizhny Novgorod", address.get("city").asText());

    ArrayNode arrayNode = (ArrayNode) actualObj.get("phoneNumbers");
    Assertions.assertEquals("88005553535", arrayNode.get(0).asText());
    Assertions.assertEquals("89205476254", arrayNode.get(1).asText());
}
```

Jackson позволяет работать с JSON как с деревом объектов

```json
{
  "firstName": "Ivan",
  "lastName": "Ivanov",
  "children" : null,
  "married": true,
  "age" : 27,
  "address": {
    "streetAddress": "First street, building 1, flat 1",
    "city": "Nizhny Novgorod"
  },
  "phoneNumbers": [
    "88005553535",
    "89205476254"
  ]
}
```

Используется, когда структура JSON заранее неизвестна,
но нет необходимости создавать отдельный POJO-объект

Задание

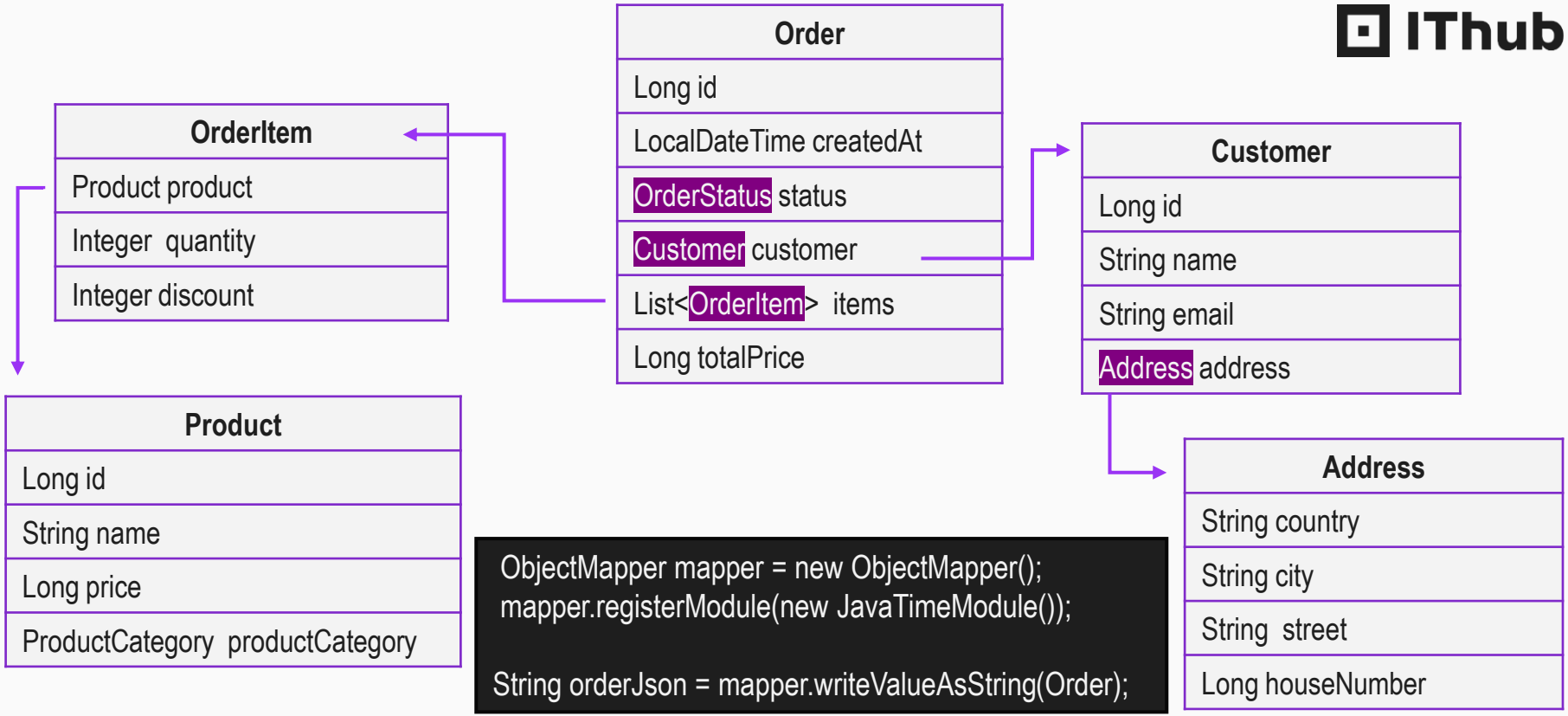
IThub

Необходимо разработать модель заказа в интернет-магазине.
Заказ содержит информацию о покупателе, список товаров, адрес доставки и дополнительные метаданные.
Требуется реализовать сериализацию Java-объекта в JSON и обратную десериализацию с помощью Jackson.

```
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <version>1.18.36</version>
</dependency>
```

```
<dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.20.0</version>
</dependency>
```

```
<dependency>
    <groupId>com.fasterxml.jackson.datatype</groupId>
    <artifactId>jackson-datatype-jsr310</artifactId>
    <version>2.22.3</version>
    <scope>compile</scope>
</dependency>
```


IThub
Order
Long id
LocalDateTime createdAt
OrderStatus status
Customer customer
List<OrderItem> items
Long totalPrice
OrderItem
Product product
Integer quantity
Integer discount
Customer
Long id
String name
String email
Address address
Product
Long id
String name
Long price
ProductCategory productCategory
Address
String country
String city
String street
Long houseNumber
ObjectMapper mapper = new ObjectMapper();
mapper.registerModule(new JavaTimeModule());
String orderJson = mapper.writeValueAsString(Order);
OrderStatus CREATED, PAID, SHIPPED, CANCELLED.
ProductCategory ELECTRONICS, BOOKS, CLOTHING, HOME